12c — Test Driven Development (TDD)

Key Concepts

- **TDD** = Development practice where tests are written *before* the code
- Core cycle: **Red** → **Green** → **Refactor**
- Forces you to define behavior before implementation
- Tests become living documentation of what the system is supposed to do

---

How It Works — Red-Green-Refactor

Step	Action	Why
Red	Write a failing test	Code doesn't exist yet — proves test catches real failures
Green	Write minimum code to make it pass	Focus on making it work, not perfect
Refactor	Clean up without breaking tests	Improve design safely

```
Write failing test (RED)
        ↓
Write minimum code (GREEN)
        ↓
Refactor cleanly (REFACTOR)
        ↓
Repeat for next requirement
```

AAA Pattern — Standard Test Structure

Every test should follow **Arrange** → **Act** → **Assert**:

```
[Fact]
public void Withdraw_ShouldReduceBalance() {
    // Arrange — set up data
    var account = new BankAccount(initialBalance: 500m);

    // Act — call the method
    account.Withdraw(100m);

    // Assert — verify the result
    Assert.Equal(400m, account.Balance);
}
```

TDD vs Writing Tests After

	TDD	Tests After
When	Before code	After code
Design influence	Tests shape the design	Tests conform to existing design
Coverage	High by default	Often incomplete
Mindset	Define behavior first	Verify behavior after

Benefits

Benefit	What it means
Built-in regression safety	Every change is covered — break something, tests catch it immediately
Forces good design	Hard-to-test code = poorly designed code — TDD surfaces this early
Living documentation	Tests describe exactly what the code does
Confidence to refactor	Restructure freely — tests tell you if you broke anything

What TDD is NOT

- Not about 100% code coverage blindly
- Not about testing every private method
- Not a substitute for integration or E2E tests
- Not always practical for exploratory/spike work

Industry Examples

Company	TDD Application
Google	Enforces TDD in core infrastructure — tests are first-class citizens
Amazon	Every microservice/Lambda has unit tests written before deployment
Microsoft	.NET Core open source repo shows test-first commits throughout

Interview Questions

Q1: What is TDD?

> Write tests before code. Follows Red-Green-Refactor: write failing test, write minimum code to pass, then refactor.

Q2: Why does the test fail first in TDD?

> Because the code doesn't exist yet. A test that never fails is useless — the first failure proves it catches real problems.

Q3: Why would a team adopt TDD?

> Built-in regression safety, forces good design (hard-to-test code = poor design), living documentation, and confidence to refactor freely.

Q4: What is the AAA pattern?

> Arrange (set up data), Act (call the method), Assert (verify the result) — standard structure for readable, consistent tests.

Q5: TDD vs writing tests after — key difference?

> In TDD, tests shape the design. Writing tests after means tests conform to existing design — often leading to incomplete coverage and untestable code.